

EOS dev8: Adapterkommunikation, Stabilität und Logiken

Stand: 02.10.2026. Produktquellstand: 0.2.0-dev.8. Basis: vollständiges dev7-Archiv, Commitbezug fad854700039dcbb172a8cf7b992666e6983fdaa; Archiv-SHA256

9cbc6d19ecfc758d6fc4fab84827faf2623a50b1a38c97fa8edbbc0cc23a4a15. Alle 6.222 Ausgangsdateien wurden gegen dessen Lieferverzeichnis geprüft.

Ergebnis und Reichweite

Der PostgreSQL-Store wurde gegen Wiederholungen alter Ereignisse gehärtet. Zusätzlich liegt ein ausführbarer, separat getesteter Erweiterungskanal mit TLS 1.3, eigenen Clientzertifikaten und serverseitigen Datenpunktrechten vor. Ein Gateway-Client kann nur ausdrücklich zugeordnete Werte lesen, eigene Messwerte melden, begrenzte Logikanforderungen schreiben und zugelassene Nachrichten mit anderen Gateway-Clients austauschen. Kein Client dieses neuen Kanals benötigt den PostgreSQL-Schlüssel des Controllers.

Dieser Stand ist ein vollständiger Entwicklungsquellstand. Er ist kein neuer installierbarer Gesamtrelease. Das enthaltene signierte 0.2.0-test.2 bleibt unverändert und enthält die dev8-Neuerungen nicht. Auf einer Anlage wurden keine Dateien geändert. Der JavaScript-Adapter wurde nicht aktiviert. Native PostgreSQL-/Systemd-Integration, getrennte OS-Konten, vollständige Adapterkompatibilität und Pi-Betrieb sind nicht abgenommen.

Der neue Kanal ergänzt die bisherige Architektur. Bestehende Controllerprozesse arbeiten weiterhin mit dem direkten PostgreSQL-Profil und dessen gemeinsamen Objects-/States-Zugängen. Eine vollständige Isolation sämtlicher vorhandener Adapter ist damit ausdrücklich noch nicht erreicht.

Abschlussprüfung: 158 gezielte Prüfungen bestanden. Die echten lokalen TLS-Verbindungen wurden mit Node 24.21.0 geprüft; der States-Speicher war in diesen Kanaltests simuliert. Die PostgreSQL-Vertragstests ersetzen keinen nativen Datenbank- oder Hardwarelauf.

Umgesetzte Änderungen

Änderung	Umsetzung	Sicherheitswirkung / Grenze
PostgreSQL-Arbeitsverbindungen	Store prüft neue Pool-Verbindungen vor ihrer Freigabe auf verschlüsselten, autorisierten TLS-1.3-Stream.	Zusätzliche Kontrolle neben verbindlicher TLS-Konfiguration. Native PostgreSQL-Nachprüfung offen.
Alte Ereignisse	Nach LISTEN wird eine Datenbank-Ereignisgrenze ermittelt; bereits zugestellte IDs werden auch nach Cacheverdrängung verworfen.	Kein erneutes Zustellen alter Referenzen nach Wiederverbindung. Voraussetzung: Schreibpfade halten den vorhandenen Transaktionslock je Datenklasse ein.
Identität je Erweiterung	CA-Prüfung, TLS 1.3, vollständiger SHA256-Zertifikatfingerprint und exakter Zertifikats-CN.	Ein gültiges Zertifikat derselben CA allein erteilt keinen Zugang. Private Schlüssel müssen je OS-Konto geschützt werden.
Serverseitige Rechte	Exakte Lese-IDs und typisierte Schreibregeln; keine Wildcardfreigabe.	Kein SQL-, Objects-, Dateisystem-, Shell-, Lizenz-, Sitzungs- oder Hostkommandozugang im neuen API.
Messwerte und Logikanforderungen	Telemetrie nur im eigenen Namespace, serverseitig ack=true; Anforderungen nur unter eos.requests.<Instanz>., ack=false.	Logiken geben Wünsche ab. Umsetzung, Prioritäten, §14a und Gerätegrenzen gehören weiterhin in den geprüften Regelkern. Es wurde kein neuer Anlagenbefehlsausführer aktiviert.
Nachrichten	Feste Empfänger-/Kommandoliste, authentifizierter Absender, Ablaufzeit, begrenzte Mailbox.	Austausch zwischen Gateway-Clients; kein transparenter Ersatz für ioBroker sendTo und keine Ausführungsbestätigung.
Neustart, Wiederholung, Widerruf	Neue Gateway-Epoche nach Neustart/Policywechsel; UUID-Wiederholungsschutz, zeitliche Gültigkeit; Policy wird erneut geprüft.	Alte Aufträge werden nicht automatisch erneut gesendet. Rechteänderung verwirft wartende Nachrichten.
Eingabekontrolle	Begrenzter JSON-Parser prüft Tiefe, Duplikatschlüssel, UTF-8, Zahlen und gefährliche Eigenschaftsnamen.	Fehlercodes ohne Nutzdaten oder Schlüssel. Nachrichtendaten brauchen zusätzlich empfängerbezogene Fachvalidierung.
Adapterkatalog	PostgreSQL-Schema und Laufzeitvalidator abgeglichen; neuer Kanal als pending erfassbar.	Eine Kanaldeklaration kann die fehlende Hostabnahme nicht umgehen. Freie JavaScript-Ausführung bleibt im bisherigen Katalog gesperrt.

Transport und Vertrauensgrenzen

- Bestandsadapter im bisherigen Controllerprofil kommunizieren über die Objects-/States-Backends mit PostgreSQL. Dort bleiben TLS 1.3, CA-/Namensprüfung und die beiden Clientzertifikate vorgeschrieben. Es sind zwei Datenklassenrollen, keine individuellen Adapterrollen.
- Eine umgebaute Erweiterung spricht über ihren eigenen mTLS-Client mit dem optionalen Gateway. Dort werden Datenpunktrechte, Wertebereiche, Absender, Empfänger und Nachrichtengültigkeit geprüft. Für States benutzt nur der Gateway-Prozess das bestehende PostgreSQL-Backend.
- Gateway-Client A und B besitzen jeweils eigene TLS-Verbindungen. Die Verschlüsselung endet am vertrauenswürdigen Gateway. Es ist **keine** Ende-zu-Ende-Verschlüsselung gegenüber dem Gateway oder einer Hostadministration.

4. Der Kanal lauscht im gelieferten Dienstentwurf ausschließlich auf 127.0.0.1:19443. Fernadapter sind nicht eingerichtet. Eine spätere Netzfreigabe benötigt eine eigene Netz-, Firewall- und Zertifikatsabnahme.
5. TLS verschlüsselt weder Daten auf SSD/SD noch automatisch Feldprotokolle wie Modbus TCP, HTTP oder unverschlüsseltes MQTT/OCPP. Diese Pfade brauchen eine eigene Bewertung; wo Geräte TLS nicht unterstützen, bleiben Netzsegmentierung und eingeschränkte Endpunkte erforderlich.

Die TLS-Kryptografie stammt aus Node/OpenSSL. Es wird kein eigener Verschlüsselungsalgorithmus eingeführt. AES-256 ist kein Ersatz für Identität, Zugriffsrechte, Patchpflege oder Prozessisolation. Die zugelassene TLS-1.3-Suite wird ausgehandelt; dev8 behauptet keine feste AES-256-Suite für jede Verbindung.

Ausfall- und Ressourcenverhalten

Der Client hat ein gemeinsames Fünfsekundenbudget für Sitzungsabruf und RPC einschließlich Verbindungsaufbau und Antwort. Er folgt keinen Redirects, fällt nicht auf Klartext zurück und wiederholt Schreibaufträge nicht automatisch. Der Gateway begrenzt zusätzlich Anfragen auf fünf Sekunden. Der PostgreSQL- Wrapper kontrolliert Frist, Widerruf und Abbruch vor und nach einem Schreibvorgang innerhalb der Transaktion; ein später Rücklauf soll zurückgerollt werden. Dieser letzte SQL-Abbruchpfad ist am nativen Server noch zu prüfen.

Bei einer unterbrochenen Antwort kann ein bereits bestätigter Datenbank-Commit nicht zurückgenommen werden. Ein Kommunikationsfehler bedeutet deshalb nicht automatisch, dass nicht geschrieben wurde. Vor einem neuen fachlichen Auftrag muss der Empfänger seinen aktuellen Zustand prüfen. Es gibt keine Zusage einer exakt einmaligen physischen Ausführung.

Grenze	Implementierter Wert
Peers / offene TCP-Verbindungen	64 / 64
Aktive Backendaufträge	32 global, 2 je Peer
Anfragenrate	20/s je Peer, kurzzeitiger Vorrat 40
RPC / Antwort	16 KiB / 64 KiB
Nachrichtennutzlast	8 KiB
Mailbox	32 je Empfänger, 256 insgesamt
Nachrichtenabruf	höchstens 16 und weniger als 60.000 Nutzlastbytes je Abruf
Wiederholungsregister	höchstens 1.024 IDs je Peer; Ablauf nach Gültigkeit
Zertifikate / Schlüssel im API-Startprofil	höchstens 64 KiB je PEM-Wert

Nachrichten sind flüchtig. Neustart, Policywechsel, Fristablauf oder ein Verlust der Abrufantwort können eine Nachricht verwerfen. `queued=true` bedeutet nur Aufnahme in eine Mailbox, ausdrücklich nicht Ausführung. Der Empfänger muss die Frist unmittelbar vor einer Aktion erneut prüfen. Wichtige Regelaufgaben dürfen daher nicht ohne Zustandsabgleich an diese Mailbox ausgelagert werden. Für Messwerte bleiben `ts`, `lc` und `q` erhalten; alte Werte werden beim Lesen nicht mit einem frischen Zeitstempel versehen. Grenzwerte, Datenalter und sichere Fallbacks müssen fachlich je Gerät festgelegt werden.

Externe Adapter: notwendiger Umbau

Nicht jeder ioBroker-Adapter muss vollständig neu geschrieben werden. Ein Adapter, der ausschließlich die unterstützte adapter-core-API nutzt, kann im bisherigen vertrauenswürdigen Controllerprofil zunächst über das PostgreSQL- Backend arbeiten. Jede konkrete Version benötigt aber Integrations- und Ausfalltests. Ein kompatibler Programmaufruf ist keine Sicherheitsfreigabe.

Für eine abgeschottete Erweiterung reicht ein Datenbankwechsel nicht. Sie muss als eigener Dienst ohne Controller-Datenbankzugang starten und einen expliziten Gateway-Client bzw. eine künftig zu entwickelnde kompatible API-Fassade verwenden.

Bisheriger Zugriff	Anpassung für den neuen isolierten Kanal
getState / getForeignState	Auf ChannelClient.readState(id) abbilden; ID serverseitig freigeben. Nur primitive Zustandswerte bzw. null werden übertragen.
Messwert setState	writeState(id, value) mit eigenem Namespace und Typ-/Wertebereich; Absender und ack setzt der Server.
Logik-Sollwert	Nur freigegebene eos.requests.<Instanz>.* schreiben; der Regelkern muss Wunsch, Prioritäten, Ablauf und technische Limits prüfen.
subscribeStates, stateChange	Im neuen API noch kein gleichwertiger Push-Vertrag. Vorläufig gezielter Abruf; vollständige Ereignis-/Alias-Kompatibilität ist ein eigener Entwicklungsschritt.
sendTo, Callback, onMessage	Explizite Empfänger-/Kommandoregeln und send/receive; Korrelation/Antwortvertrag fachlich ergänzen. Kein sendToHost-Tunnel.
Objects, Views, Dateien, Zertifikate	Im Gateway nicht verfügbar. Benötigte nicht geheime Metadaten als enges, geprüftes API ergänzen. Kein pauschaler Zugriff auf system.*.
Direkter Redis-/SQL-Zugriff	Entfernen. Weder DB-Schlüssel noch DB-Bibliothek in die Erweiterung übernehmen.
npm-/Git-Nachinstallation	Entfernen bzw. in einen geprüften Build verlagern; exakte Versionen, Lockfile, SBOM und signiertes Herstellerpaket verwenden.
exec, Shell, native Tools	Entfernen oder nur separat geprüfte feste Hilfsdienste verwenden. Kein allgemeiner Shell-Proxy im Gateway.
LAN-/Geräteverbindung	Ziele und Protokolle einzeln deklarieren; TLS prüfen, Fristen und Antwortgrößen begrenzen. Ein lokales Gateway verschlüsselt den Geräteanschluss nicht.

Aufnahmeablauf: konkrete Version und vollständige Quellen sichern; API-/Netz- und Rechtebedarf erfassen; Umbau; reproduzierbarer Build und SBOM; Negativ- und Kompatibilitätstests; eigenes OS-Konto und eigenes Zertifikat; Hostprüfung; isolierter Pi-Test; erst danach Eintrag mit belegter Testfreigabe. Signaturprüfung und geschützter Installationspfad bleiben Teil der noch offenen Hostintegration.

JavaScript-Adapter: Befund und Zielbild

Geprüfte Quelle: offizielles npm-Artefakt iobroker.javascript@10.3.0, am 02.10.2026 abgerufen. SHA512 wurde gegen die npm-Metadaten geprüft. Das Paket wurde weder installiert noch ausgeführt. Die folgenden Befunde sind eine Quellprüfung des ausgelieferten Codes, kein nachgewiesener Angriff auf EOS und keine Zuordnung zu einer CVE.

Befund in 10.3.0	Bedeutung für EOS
build/main.js stellt mods.child_process, net, http, https und weitere Module bereit.	Der Schalter enableExec=false begrenzt den Komfortaufruf exec, aber nicht das gesamte Modulsystem.
build/lib/sandbox.js liefert vorhandene Module aus mods und besitzt weitere require-Rückfälle.	Ein Skript darf nicht als abgeschottet gelten, nur weil einzelne API-Schalter ausgeschaltet sind.
runInNewContext in build/main.js	Node dokumentiert node:vm ausdrücklich nicht als Sicherheitsgrenze für nicht vertrauenswürdigen Code. Auch Node-Permissions sind allein keine solche Grenze.
native.enableSecrets=true und SECRETS-Bereitstellung	Zentrale Geheimnisse dürfen nicht pauschal in den Logikprozess gelangen. Für EOS standardmäßig deaktivieren und benötigte Werte einzeln freigeben.
installLibraries() mit konfigurierbaren Modulen bzw. URLs	Kein Paketnachladen aus Skript-/Adapterkonfiguration im Zielfprofil. Erlaubte Module werden im Herstellerbuild festgelegt.
Skriptänderung, Debugger, Spiegelverzeichnis und Netzwerkfunktionen	Separate Rechte und Prozessgrenzen nötig. Ein Skriptbearbeiter erhält praktisch Codeausführungsrechte innerhalb seines Logikbereichs.

Empfohlen ist ein **EOS-Fork mit getrenntem Logikdienst**: Der Editor darf als kontrollierte Oberfläche erhalten bleiben; die Ausführung findet je Vertrauensgruppe in einem eigenen, begrenzten OS-Prozess statt. Skripte derselben Instanz teilen weiterhin deren Rechte. Unterschiedlich vertrauenswürdige Skripte benötigen getrennte Instanzen/Identitäten, nicht nur verschiedene Dateinamen.

Der Fork muss das direkte mods-Objekt, freie require-/Import-Rückfälle, child_process, Hostbefehle, beliebigen Netzwerk-/Dateizugriff, npm-Nachladen und zentrale Secrets aus dem Skriptvertrag entfernen. Freigegebene getState-, Anforderungs-, Zeitplan- und Logfunktionen bekommen enge Gateway-Entsprechungen. Blockly ist ebenfalls Codeerzeugung und benötigt dieselbe Ausführungsgrenze. Ein Verbot einzelner JavaScript-Wörter wäre keine tragfähige Isolation.

Skriptfreigabe und Rollen: Endnutzer verändern nur vorgegebene Parameter; Installateure verwenden geprüfte Vorlagen und freigegebene Datenpunkte; Serviceadministration prüft, versioniert und aktiviert neue Skripte. Für Leistungsbegrenzung, Schutzfunktionen und Failsafe bleibt der Regelkern zuständig. Ein Skriptausfall oder eine Endlosschleife darf den Controller nicht blockieren.

Die gelieferten systemd-Dateien sind als .example gekennzeichnete Entwürfe. Sie sehen eigene Konten, geschützte Dateien, individuelle Credentials, CPU-/Speicher-/Prozessgrenzen und lokalen Netzwerkzugang vor. Das bestehende Controllerkonto darf nicht mitbenutzt werden. Der wirksame Schutz gegen Lesen fremder Schlüssel, /proc-Zugriff, Netzwerkumgehung und Ressourcenerschöpfung muss auf Debian 13 nachgewiesen werden. Die Vorlagen sind nicht automatisch installiert und starten keinen unveränderten JavaScript-Adapter.

Ein sicher integrierter JavaScript-Adapter ist damit noch nicht fertiggestellt. Der neue Kommunikationsbaustein und die Umbauanforderungen liegen vor; der Worker-/Editorumbau und seine Isolationstests bleiben Arbeit vor Aktivierung. Die Aussage „ohne Sicherheitsbedenken“ lässt sich seriös nicht zusagen.

Prüfbelege und offene Arbeit

Der Abschlusslauf mit Node 24.21.0 und dessen OpenSSL 3.5.8 bestand: **158 Prüfungen** (35 Kanal-/TLS-Prüfungen, 27 Katalogprüfungen, 91 PostgreSQL-Vertragsprüfungen, 5 JSON-Schemaprüfungen), keine fehlgeschlagenen oder übersprungenen Tests. Die beiden neuen/aktualisierten CycloneDX-Nachweise wurden separat erfolgreich gegen das vorhandene Schema validiert. Bei 13 tatsächlich vorliegenden öffentlichen Paketen des PostgreSQL-Node-Treibers lieferte npm am Prüftag keine Advisories zurück. Das ist eine begrenzte Meldungsabfrage, kein vollständiger Schwachstellen- oder Produktnachweis.

Aktuelle Zahlen und Befehle stehen in `reports/integration/adapter-channel/verification-summary.json`; unveränderte Rohbelege unter `raw/`, Hashbindung unter `tested-sources.json`. Die Tests unterscheiden echte lokale TLS-/HTTP-Kommunikation von simuliertem States-Speicher und PostgreSQL-Vertragsprüfungen mit Testdoubles. Die früheren 558 Testeinträge aus dev7 werden nicht als neue Tests gezählt.

Die Quellprüfung hat außerdem einen veralteten JSON-Schema-Vertrag gefunden: Die Katalogbibliothek kannte PostgreSQL bereits, das Schema noch nicht. Beides ist jetzt abgeglichen; für den neuen Gatewaykanal bleiben Freigaben bis zur Hostintegration gesperrt.

Befundkennung	Stand / Priorität	Nächster erforderlicher Nachweis
EOS-PG-REPLAY-20261002	Code korrigiert; mittel; native Nachprüfung offen	Mehrere echte PG-Clients, Verbindungsabbruch und Meldungsreihenfolge.
EOS-PG-TLSPPOOL-20261002	Zusätzliche Streamkontrolle; Härtung	Echte PG-Poolverbindung mit gültigen/ungültigen Zertifikaten.
EOS-ADMISSION-SCHEMA-20261002	Schema-/Codeabweichung korrigiert	Passende positive/negative Schema- und Katalogtests im Bericht.
EOS-CHANNEL-HOST-20261002	Offen; hoch; Aktivierungssperre	Eigene OS-Konten, Credentialbereitstellung, Start-/Installationspfad, Zugriffstests.
EOS-JS-TRUST-20261002	Offen; hoch; JavaScript bleibt inaktiv	Fork, separater Worker, Rechte-/Flucht-/Lasttests und Skriptfreigabe.
EOS-PG-INTEGRATION-20261001	Bestehende Sperre bleibt offen	PostgreSQL 17.11, Controller 7.2.2 und Pi/Systemd gemeinsam betreiben und prüfen.
Zertifikatslebenszyklus / Wiederherstellung	Offen; vor Produktfreigabe	Rotation, Widerruf, Ablauf, Uhrzeitfehler, Backup/Restore einschließlich Policy.

Verantwortlicher Bereich: NexoWatt Entwicklung; Abnahme durch NexoWatt am festgelegten Zielsystem. Die nächsten Schritte sind zuerst ein nativer Gateway-/PostgreSQL-Lauf, danach die Hostisolation und ein einzelner umgebauter Testadapter. Anschließend folgt der JavaScript-Worker. Für jeden Schritt sind Rückfall, Ausfallverhalten und Anlagenlimits gesondert zu prüfen.

Es wurde kein unabhängiger Penetrationstest durchgeführt. Dieser Bericht ist kein CRA-/IEC-Konformitätsnachweis und enthält keine Produktfreigabe. Die bestehenden OS-Sicherheitsupdatefunktionen bleiben im Quellpaket erhalten; ihre früheren Tests werden nicht als neue Abnahme ausgegeben.

Offizielle Quellen

- Node vm: <https://nodejs.org/api/vm.html>
- Node Permission Model: <https://nodejs.org/api/permissions.html>
- PostgreSQL Zertifikatsauthentifizierung: <https://www.postgresql.org/docs/17/auth-cert.html>
- PostgreSQL NOTIFY und Sichtbarkeit: <https://www.postgresql.org/docs/current/sql-notify.html>
- ioBroker JavaScript Quellprojekt: <https://github.com/ioBroker/ioBroker.javascript>
- Geprüftes npm-Artefakt: <https://registry.npmjs.org/iobroker.javascript/-/iobroker.javascript-10.3.0.tgz>

Quellen wurden am 02.10.2026 abgerufen. Die npm-Metadaten und das geprüfte Upstream-Artefakt liegen als nicht aktivierter Prüfgegenstand im Berichtspfad.